

SERVICE ACTUEL · NOTIFICATION-CENTER

Notifications, abonnements et diffusion temps réel

13 capacités analysées

services/notification-center · Kotlin/Spring Boot, modules domaine et infrastructure ; PostgreSQL/JPA/Flyway ; Kafka/Protobuf pour les événements externes et JSON pour la livraison interne ; JavaMail/SMTP, FreeMarker ; netty-socketio et JWT. Toutes les briques JVM doivent disparaître dans la cible.

Maintenir les destinataires et leurs abonnements, transformer les événements métier en notices regroupées, servir une boîte personnelle, remettre les emails prévus et diffuser les changements de collecte, flow et cluster aux interfaces connectées.

Lecture de code

Pas de qualification runtime nouvelle

Domaines cible : Notifications, IAM, Audit, Egress, Flows, Datasets, Runtimes, Frontends, Migration.

Périmètre de lecture

- AGENTS.md du monorepo et du composant, règles Helm et contrat d'authentification des sockets.
- Les deux controllers NotificationController et SubscriptionController, tous leurs DTO, les agrégats et handlers de notification/souscription, les repositories et mappings persistants.
- NotificationConsumer, lamUserEventConsumer, DataDevEventConsumer, SendNotificationEventConsumer et SynchronousCommunicationConsumer ; convertisseurs Protobuf et messages JSON internes.
- Regroupement des accumulations, lamDeliveryAuthorization, rendus mail et leurs trois templates, émetteur INAPP, TokenAuthService, WsAuthorizationService, SocketIoService et gate du protocole.
- Outbox, idempotence, dead-letter, gestion d'erreur Kafka, santé des listeners, sécurité HTTP, migrations SQL et de code, propriétés applicatives et dépendances Gradle.
- Chart : deployment, service, réseau, secrets référencés, probes, volumes et autoscaling ; inventaire des tests de contrats et tests wire, sans exécution.

Objets et cycle de vie

- Subscriber : UUID compatible avec le sujet JWT historique, nom/prénom, locale, médias et abonnements ; désactivation logique et réactivation possibles. IAM est la source d'identité, le service en maintient un miroir.
- Media : INAPP ou EMAIL et destination ; les identifiants de médias sont conservés lors des modifications d'abonné.
- Subscription : association abonné/entité avec filtres d'actions et médias. EntityActionTypeDAO est global par nom d'action ; la portée réelle de préférences médias différentes doit être caractérisée.
- Accumulation : action, ressource origine, ressource notifiée, destinataire, contenus JSON/mail, date et UUID de message optionnel ; supprimée après réduction du groupe.
- Notification : résultat regroupé avec UUID, destinataire, ensemble d'entités, action, dates de création/lecture, contenus et canaux. La boîte INAPP conserve ce résultat en PostgreSQL.
- OutboxEvent : payload et topic à transmettre, date d'envoi et tentatives ; ProcessedMessage : clé de groupe/UUID ; DeadLetterEvent : payload, origine transport et cause d'échec.
- Session Socket.IO : sujet authentifié, token expirant, namespaces et rooms en mémoire ; la connexion et ses messages ne sont pas un historique durable.

- Migration de code : tâche enregistrée en base et exécutée au démarrage ; distincte de l'historique Flyway.

Dépendances du service

- IAM et IdP : JWT, sujets historiques, événements utilisateurs et broker can_view ; les headers HTTP de proxy ne sont fiables que dans le mode explicitement configuré.
- Producteurs métier : contrats InitNotificationEvent et SynchronousCommunicationEvent de packages/proto ; data-collect publie notamment demandes d'accès, échecs de collecte et résultats CHECK_FILE.
- Consommateurs des événements de souscription : changements réémis via outbox en Protobuf ; les migrations de code rejouent certaines souscriptions.
- Kafka : transport entrant, livraison interne en deux groupes par canal et communication synchrone ; le bus JVM actuel ne satisfait pas la cible finale.
- PostgreSQL : abonnés, préférences, accumulations, boîte, outbox, marqueurs et quarantaine ; sauvegarde et restauration doivent couvrir cet ensemble.
- SMTP : remise d'emails HTML ; le nom SendinBlue de la classe ne prouve pas une intégration API fournisseur.
- APISIX et frontends : routes HTTP et Socket.IO ; la route socket laisse l'authentification au service ; rooms et noms d'événements constituent des contrats clients.
- DataDevEventConsumer décode aussi des changements de projet et de membres dont les handlers lus journalisent les données sans modifier les abonnements ; ne pas leur attribuer une synchronisation absente.

Fonctionnalités et compatibilité à préserver

NC-01 Cible non qualifiée

Synchroniser les abonnés et leurs coordonnées depuis l'identité

COMPOTEMENT OBSERVÉ

Un abonné est identifié par le sujet utilisateur et conserve nom, prénom, langue, médias et abonnements. Les événements historiques Subscriber créent, modifient ou désactivent ce miroir. Le nouveau consumer IAM exige un messageUid pour la déduplication ; il utilise le keycloakId comme identifiant compatible avec les sujets JWT. Il peut créer, mettre à jour, réactiver ou désactiver un abonné selon active. La suppression utilise un indicateur séparé et les messages d'offboarding sont ignorés car ce service ne possède pas les ressources à réassigner. Le consumer IAM démarre seulement si sa feature est activée. Les langues sont FR, EN et DE, avec normalisation des sous-tags et repli EN. La mise à jour conserve les identifiants de médias.

UTILISATEURS ET RÔLES

IAM, utilisateurs destinataires, opérateurs de migration d'identité.

INTERFACES DANS LE CODE

NotificationConsumer : branche SUBSCRIBER ; iamUserEventConsumer ;
SubscriberCreateOrUpdateCommandHandler ; SubscriberDeleteCommandHandler.

DONNÉES ET ÉTATS

subscriber avec suppression logique ; medias EMAIL/INAPP ; noms et locale ; processed_message pour le nouveau consumer IAM.

DÉPENDANCES ET ÉCHANGES

Événements utilisateurs historiques et IAM, Protobuf, PostgreSQL ; sujets JWT partagés avec le frontend.

COMPORTEMENT CIBLE PROPOSÉ

Notifications maintient un miroir Rust minimal de IAM avec sujet canonique, tenant, locale et préférences. IAM reste autoritaire pour l'activation. Les anciens sujets deviennent des alias explicites et les suppressions ont une politique de rétention distincte des reçus de livraison.

MIGRATION DE CETTE CAPACITÉ

Rejouer les utilisateurs IAM depuis l'origine dans un consumer isolé, comparer les UUID et les adresses, puis choisir une source unique. Réconcilier les abonnés soft-deleted avant d'activer les suppressions. Remplacer la référence Keycloak en même temps que l'IdP JVM.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Le replay de la même création conserve l'abonné et les médias. Une mise à jour fr-CA devient FR, une langue inconnue EN ; une désactivation retire les futures livraisons et une réactivation restaure le sujet sans dupliquer ses abonnements.

Références de l'analyse

[monorepo · services/notification-center/app/notification-center-domain/src/main/kotlin/com/cleyrop/domain/subscription/command/SubscriberCreateOrUpdateCommandHandler.kt:27](#) Mise à jour et réactivation du miroir.

[monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/common/consumer/IamUserEventConsumer.kt:60](#) Transaction, UUID et types d'événements.

[monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/common/consumer/IamUserEventConsumer.kt:124](#) Suppression sous indicateur et choix du keycloakId.

[monorepo · services/notification-center/app/notification-center-domain/src/main/kotlin/com/cleyrop/domain/subscription/aggregate/Subscriber.kt:25](#) Langues, champs et conservation des médias.

NC-02 **Cible non qualifiée**

Créer les abonnements personnels aux échecs de dataset et de flow**COMPORTEMENT OBSERVÉ**

La création REST s'applique à l'utilisateur authentifié, sans identifiant de destinataire fourni par le client. Le DTO exige une entité UUID, un type DATASET ou DATAFLOW et une liste non vide d'actions cohérentes avec ce type. Les trois actions exposées sont DATAFLOW_EXECUTION_FAILED, DATASET_COLLECT_FAILED et DATASET_QUALITY_FAILED. Les médias acceptés sont INAPP et EMAIL, sélectionnés tous deux par défaut. Le domaine ajoute les filtres absents et émet un événement de création d'abonnement. Un abonné absent entraîne un résultat nul puis une erreur 500 du controller. Le mapping persistant des médias est porté par EntityActionTypeDAO identifié globalement par le nom d'action ; la capacité d'avoir des préférences différentes selon deux abonnés doit donc être caractérisée par un test ciblé.

UTILISATEURS ET RÔLES

Utilisateurs de l'interface et clients authentifiés qui surveillent datasets et flows.

INTERFACES DANS LE CODE

POST /subscriptions ; SubscriptionCreateDTO ; EntityActionTypeDTO ; EntityTypeDTO.

DONNÉES ET ÉTATS

Subscriber, Subscription(entityId), EntityActionType et mediaTypes ; association entre souscription et type d'action.

DÉPENDANCES ET ÉCHANGES

Principal issu du JWT ou du proxy de confiance ; contrôle de route IAM à l'entrée ; outbox d'événements de souscription.

COMPOTEMENT CIBLE PROPOSÉ

Notifications expose un catalogue versionné des actions réellement livrables et des canaux par action. Les préférences sont stockées par tenant, sujet, ressource, action et canal. IAM contrôle le droit de s'abonner à la ressource et le droit de recevoir son contenu.

MIGRATION DE CETTE CAPACITÉ

Importer les abonnements avec un relevé des médias réellement persistés. Définir la valeur cible des préférences ambiguës avant conversion. Garder les trois actions historiques, annoncer séparément les nouvelles possibilités.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Deux utilisateurs choisissent des canaux différents pour la même action sans se modifier mutuellement. Une action de flow sur une entité DATASET est refusée. Un utilisateur sans miroir obtient une erreur métier définie plutôt qu'un abonnement fantôme.

Références de l'analyse

monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/subscription/controller/SubscriptionController.kt:49 Création limitée au principal courant.

monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/subscription/controller/dto/SubscriptionCreateDTO.kt:11 Validation du type et médias par défaut.

monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/subscription/controller/dto/EntityActionTypeDTO.kt:7 Exactement trois actions REST.

monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/subscription/repository/dao/EntityActionTypeDAO.kt:17 Clé d'action globale et colonne média.

NC-03 **Cible non qualifiée**

Recevoir les actions métier et les abonnements créés par les producteurs

COMPOTEMENT OBSERVÉ

Le consumer externe decode InitNotificationEvent et distingue ACTION_RESOURCE, SUBSCRIPTION et SUBSCRIBER. Les commandes d'abonnement produites par d'autres services peuvent nommer un ou plusieurs utilisateurs, ressources, actions et médias ; la conversion combine les ressources et les actions. La suppression événementielle retire les abonnements à l'entité pour l'utilisateur indiqué, ou pour tous les utilisateurs selon le message. Le contrat Protobuf définit 27 actions actives couvrant datasets, projets, datasources, flows, applications et demandes Git ; cette énumération ne prouve ni un producteur vivant ni un template email pour chaque action. Les créations et suppressions d'abonnements sont réémises via outbox en Protobuf. Certaines migrations retirent des anciennes souscriptions, notamment GIT_MR_REQUESTED, bien que ce nom reste au contrat.

UTILISATEURS ET RÔLES

Services producteurs de demandes d'accès, collectes, flows, projets et applications ; utilisateurs destinataires choisis par ces services.

INTERFACES DANS LE CODE

InitNotificationEvent : ACTION_RESOURCE/SUBSCRIPTION/SUBSCRIBER ; SubscriptionEventHandlerKafkaImpl ; EnumEntityTypeActionType.

DONNÉES ET ÉTATS

Abonnements automatiques et événements

SubscriberCreateSubscriptionEvent/SubscriberDeleteSubscriptionEvent ; noms d'action conservés en texte dans le domaine.

DÉPENDANCES ET ÉCHANGES

packages/proto ; bus Kafka ; services producteurs dont data-collect et data-dev ; migrations Flyway et migrations de code.

COMPORTEMENT CIBLE PROPOSÉ

Notifications maintient un registre d'actions, leur schéma de payload, les acteurs producteurs autorisés et les canaux disponibles. Une commande automatique indique sa raison, son origine et son idempotence ; elle ne permet pas à un producteur non autorisé d'abonner arbitrairement un autre tenant.

MIGRATION DE CETTE CAPACITÉ

Recenser les producteurs effectifs pour les 27 actions avant de retirer les noms obsolètes. Convertir les contrats Protobuf sans réutiliser leurs numéros réservés. Conserver les événements de souscription nécessaires aux services qui en maintiennent une projection.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Rejouer une création automatique puis sa suppression ciblée sans toucher aux autres utilisateurs. Un ancien événement connu reste décodable ; une action retirée suit une règle explicite de rejet ou d'archivage et ne disparaît pas silencieusement.

Références de l'analyse

[monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/common/consumer/NotificationConsumer.kt:48](#)
Dispatch des trois familles d'événements.

[monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/common/consumer/convertter/InitNotificationEventProtoConverter.kt:58](#) Conversion des abonnements et médias.

[monorepo · packages/proto/src/main/proto/notificationcenter/entity_action_type.proto:8](#) Catalogue complet des actions et numéros réservés.

[monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/subscription/service/producer/SubscriptionEventHandlerKafkaImpl.kt:18](#) Publication des changements de souscription via outbox.

NC-04 **Cible non qualifiée**

Lister, vérifier et supprimer les abonnements personnels**COMPORTEMENT OBSERVÉ**

Les routes de lecture renvoient les UUID d'entités pour une action ou regroupent les abonnements par liste d'actions. La vérification en lot transforme les requêtes en Set puis renvoie une liste de booléens ; deux requêtes identiques peuvent donc ne produire qu'un seul résultat. La suppression REST sépare la liste de couples reçus en une liste d'entités et une liste d'actions ; le filtre du domaine retire tout abonnement dont

l'entité appartient à la première et l'action à la seconde. Cette lecture établit une sémantique de produit cartésien à caractériser pour un lot hétérogène, sans conclure à un incident observé. Les lectures sont toujours attachées au principal courant.

UTILISATEURS ET RÔLES

Interfaces de suivi, tableaux d'abonnement et utilisateurs authentifiés.

INTERFACES DANS LE CODE

GET /subscriptions/entities?entityActionType=... ; GET /subscriptions?entityActionType=... ; POST /subscriptions/check_bulk ; DELETE /subscriptions.

DONNÉES ET ÉTATS

Subscriptions du sujet courant ; requêtes bulk dédoublées ; modifications persistées puis événements de suppression.

DÉPENDANCES ET ÉCHANGES

Repository des abonnés et outbox ; frontend qui associe potentiellement les réponses bulk par position.

COMPORTEMENT CIBLE PROPOSÉ

Notifications spécifie une correspondance explicite requête/résultat par clé de corrélation et une suppression par couples exacts. Les opérations par ensemble ou par produit cartésien, si nécessaires, deviennent des commandes nommées distinctes.

MIGRATION DE CETTE CAPACITÉ

Capturer le comportement attendu des clients pour les doublons et les lots croisés. Ajouter des routes versionnées si la correction change le contrat visible. Dédupliquer les données historiques avec un rapport avant réimport.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un lot contenant deux fois le même couple retourne deux résultats corrélés si le contrat conserve les positions. Supprimer A/collect et B/quality ne retire pas A/quality. Les réponses ne révèlent pas les abonnements d'un autre sujet.

Références de l'analyse

monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/subscription/controller/SubscriptionController.kt:93 Séparation des entités/actions pour suppression.

monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/subscription/controller/SubscriptionController.kt:156 Conversion en Set du bulk.

monorepo · services/notification-center/app/notification-center-domain/src/main/kotlin/com/cleyrop/domain/subscription/aggregate/Subscriber.kt:99 Prédicat de suppression par appartenance aux deux listes.

monorepo · services/notification-center/app/notification-center-domain/src/main/kotlin/com/cleyrop/domain/subscription/query/FindSubscriptionBulkQueryHandler.kt:19 Résultats de vérification par requête distincte.

NC-05 **Cible non qualifiée**

Choisir les destinataires et contrôler leur accès au contenu

COMPORTEMENT OBSERVÉ

Une action métier porte une ressource d'origine et une ressource à notifier. Le handler recherche les abonnés correspondant à l'une ou à l'autre pour cette action et les déduplique par sujet. Sans abonné, il ne crée rien et le consumer compte le rejet. Pour les trois actions librement souscriptibles, `IamDeliveryAuthorization` vérifie `can_view` sur les ressources auxquelles l'utilisateur est effectivement abonné ; au moins une réponse `ALLOW` suffit. Les modes `OFF`, `SHADOW` et `ENFORCE` respectivement désactivent, observent ou imposent ce contrôle ; en `ENFORCE`, une panne IAM retient la notification. Les autres familles sont laissées à la sélection du producteur. Le `messageUuid` entrant, quand il est valide, exclut les abonnés qui ont déjà une accumulation portant cet UUID ; les messages historiques sans UUID restent acceptés.

UTILISATEURS ET RÔLES

Services producteurs, abonnés, IAM et opérateurs du déploiement progressif des droits.

INTERFACES DANS LE CODE

`AccumulationCreateCommandHandler` ; `IamDeliveryAuthorization.mayReceive` ; `NotificationConsumer.consume`.

DONNÉES ET ÉTATS

Une Accumulation par destinataire éligible, action et message ; `messageUuid` optionnel ; compteurs sans abonné et décisions IAM.

DÉPENDANCES ET ÉCHANGES

Broker IAM `can_view` pour `DATASET/DATAFLOW` ; configuration `ws.authz.mode` ; abonnements à la ressource origine ou notification.

COMPORTEMENT CIBLE PROPOSÉ

Notifications contrôle toutes les familles selon une politique explicite. Le contrôle avant effet prend la décision IAM courante et masque les champs que le destinataire ne peut plus lire. La clé `source/message/destinataire` reste durable au-delà de la durée d'une accumulation.

MIGRATION DE CETTE CAPACITÉ

Passer les décisions en shadow sur toutes les familles, analyser les destinataires divergents, puis imposer les politiques. Attribuer un message ID aux producteurs historiques. Définir la gestion des notifications de suppression dont la ressource n'est plus consultable.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un abonné privé d'accès avant émission ne reçoit pas de contenu protégé. Une indisponibilité IAM en `ENFORCE` garde le travail rejouable. Le replay d'un message après vidage des accumulations reste sans seconde notification dans la cible.

Références de l'analyse

[monorepo · services/notification-center/app/notification-center-domain/src/main/kotlin/com/cleyrop/domain/notification/command/AccumulationCreateCommandHandler.kt:56](#) Sélection origine/notification et absence de destinataire.

[monorepo · services/notification-center/app/notification-center-domain/src/main/kotlin/com/cleyrop/domain/notification/command/AccumulationCreateCommandHandler.kt:72](#) Autorisation puis déduplication par accumulations existantes.

[monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/notification/service/IamDeliveryAuthorization.kt:38](#) Modes et comportement en erreur.

[monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/notification/service/IamDeliveryAuthorization.kt:103](#) Périmètre limité aux trois familles REST.

NC-06 **Cible non qualifiée**

Regrouper les événements proches en une notification

COMPORTEMENT OBSERVÉ

L'orchestrateur traite périodiquement les groupes d'accumulation dont le dernier événement est assez ancien. Le délai du scheduler est borné à cinq secondes au minimum ; la requête d'éligibilité considère la fenêtre de silence configurée. Le groupe associe action, ressource d'origine, abonné et valeur `trigger_lastname` utilisée comme identité de déclencheur. Le service fusionne les `ActionObjects` JSON distincts, garde le premier sujet mail et ajoute les noms du destinataire aux objets réservés au mail. Pour `mailContent`, il prend le premier contenu non vide. Il recharge les abonnés et leurs abonnements pour calculer les canaux actuels. Après publication de la notification, il supprime les accumulations du groupe ; un groupe en erreur est conservé et un destinataire disparu est écarté. Le parcours lu n'expose pas de revendication verrouillée ni de transaction englobant sélection, publication et suppression, ce qui appelle un scénario de concurrence et de crash.

UTILISATEURS ET RÔLES

Destinataires recevant des événements répétés et opérateurs qui règlent le regroupement.

INTERFACES DANS LE CODE

`AccumulationOrchestrator` ; `DecreaseAccumulationCommandHandler` ; `AccumulatorServiceImpl` ; `AccumulationJPA.findAvailable`.

DONNÉES ET ÉTATS

accumulation, vue `accumulation_grouped`, Notification nouvellement identifiée ; groupes supprimés après succès.

DÉPENDANCES ET ÉCHANGES

Horloge, PostgreSQL, snapshots actuels des abonnés et outbox interne de livraison.

COMPORTEMENT CIBLE PROPOSÉ

Notifications possède un moteur Rust de regroupement avec fenêtre et clé documentées, claim à bail, identifiant de groupe stable et sortie atomique vers l'outbox. Les payloads sont versionnés et bornés en taille ; l'identité du déclencheur est un champ typé.

MIGRATION DE CETTE CAPACITÉ

Reproduire les regroupements existants sur un corpus anonymisé. Décider si le premier contenu mail est le comportement voulu. Reprendre les accumulations en attente en préservant la frontière temporelle de chaque groupe.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Deux workers traitent le même groupe sans produire deux identifiants de notification. Une panne entre création et suppression reprend le même résultat. Des actions ou déclencheurs distincts ne se mélangent pas et un événement continu suit une borne de latence définie.

Références de l'analyse

[monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/notification/service/orchestrator/AccumulationOrchestrator.kt:13](#) Cadence de traitement.

[monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/notification/service/AccumulatorServiceImpl.kt:21](#) Clé de regroupement et fusion des contenus.

[monorepo · services/notification-center/app/notification-center-domain/src/main/kotlin/com/cleyrop/domain/notification/command/DecreaseAccumulationCommandHandler.kt:31](#) Traitement, succès/suppression et conservation des erreurs.

monorepo · services/notification-center/app/notification-center-
 infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/notification/repository/jpa/AccumulationJPA.kt:2
 4 Sélection des groupes éligibles.

NC-07 **Cible non qualifiée**

Acheminer séparément les livraisons INAPP et EMAIL

COMPORTEMENT OBSERVÉ

Une notification préparée est sérialisée en message JSON interne, avec son ID, les médias calculés et un snapshot de l'abonné. Deux groupes de consommation distincts assurent EMAIL et INAPP. Chaque branche vérifie le canal, puis un marqueur processed_message propre au groupe et à l'ID de notification. Le handler SMTP peut être désactivé globalement ; dans ce cas il retourne avant le marquage. La branche INAPP persiste la notification et émet un événement de communication synchrone via outbox. Les méthodes de consommation sont transactionnelles pour les écritures PostgreSQL. L'appel SMTP reste un effet externe à cette transaction : le code ne démontre pas un envoi exactement une fois après crash. L'événement de lecture d'une notification est seulement journalisé par le handler d'événements.

UTILISATEURS ET RÔLES

Destinataires, interfaces connectées et opérateurs SMTP.

INTERFACES DANS LE CODE

SendNotificationEventConsumer.consumeEmail/consumeInApp ; NotificationKafkaMessage ;
 InAppNotificationEmitterImpl ; NotificationEventHandlerKafkaImpl.

DONNÉES ET ÉTATS

processed_message indexé par groupe et messageUuid ; notification persistée pour INAPP ; outbox de communication synchrone ; snapshot d'abonné embarqué.

DÉPENDANCES ET ÉCHANGES

Bus Kafka interne JSON, PostgreSQL, serveur SMTP, Socket.IO indirect via événements Protobuf.

COMPORTEMENT CIBLE PROPOSÉ

Notifications orchestre des effets Rust par canal avec reçu durable, état prepared/sending/accepted/failed/unknown et tentative corrélée. PostgreSQL est autoritaire pour ces reçus. La notification référence le run et l'édition approuvée quand elle annonce une publication ; elle ne devient pas la preuve de publication.

MIGRATION DE CETTE CAPACITÉ

Importer les notifications, marqueurs et travaux non livrés. Remplacer les snapshots trop anciens par des références réévaluées avant effet. Définir un traitement manuel ou idempotent des envois SMTP dont l'acceptation est incertaine.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Rejouer le même message produit une seule entrée de boîte INAPP. Une panne après acceptation SMTP et avant commit donne un état incertain explicite dans la cible. Couper SMTP ne bloque pas INAPP et ne fait pas croire qu'un email a été remis.

Références de l'analyse

monorepo · services/notification-center/app/notification-center-
 infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/common/consumer/SendNotificationEventConsumer.kt
 :35 Transactions, sélection des canaux et marqueurs.

monorepo · services/notification-center/app/notification-center-
 infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/notification/service/notification/inapp/notificationemitter/InAppNotificationEmitterImpl.kt:35 Persistance puis émission de l'événement temps réel.

[monorepo](#) · [services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/notification/service/producer/NotificationEventHandlerKafkaImpl.kt:19](#) Outbox des nouvelles notifications et lecture uniquement journalisée.

NC-08 **Cible non qualifiée**

Consulter sa boîte de notifications et marquer la lecture

COMPORTEMENT OBSERVÉ

GET /notifications retourne les notifications du principal courant, paginées et triées ; le tri par défaut est createdAt décroissant. Un abonné inexistant obtient une page vide. PUT /notifications/read exige une liste d'UUID non vide et reçoit un readAt fourni par le client, par défaut l'instant courant. Le repository filtre les UUID par leur propriétaire avant de modifier les dates de lecture, donc des UUID étrangers ne suffisent pas à toucher la boîte d'autrui. Le DTO expose l'action, les dates, le propriétaire/déclencheur et un payload structuré par type d'objet. Il retire les objets réservés au mail et certains noms techniques du destinataire. La suppression HTTP de notification n'est pas exposée.

UTILISATEURS ET RÔLES

Utilisateurs connectés, cloche de notification et centre de notifications frontend.

INTERFACES DANS LE CODE

GET /notifications?page=...&size=...&sort=... ; PUT /notifications/read.

DONNÉES ET ÉTATS

Notification : UUID, subscriberId, entityIds, action, createdAt, readAt, contenus JSON/mail, médias ; suppression logique présente dans le mapping DAO.

DÉPENDANCES ET ÉCHANGES

Principal utilisateur ; PostgreSQL ; objets de présentation partagés avec les événements Socket.IO.

COMPORTEMENT CIBLE PROPOSÉ

Notifications sert une boîte Rust par tenant/sujet avec pagination stable, curseur et total non lu. La date serveur fait foi pour la lecture ; une date client éventuelle reste séparée. Une politique de rétention contrôle archivage et suppression sans effacer les reçus d'effets requis.

MIGRATION DE CETTE CAPACITÉ

Conserver les UUID, horodatages et payloads lisibles, documenter les tris acceptés et la conversion des propriétaires. Caractériser les cas readAt fourni et notifications supprimées avant de figer le contrat cible.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une lecture paginée ne mélange pas les utilisateurs. Un lot comprenant un UUID étranger ne le modifie pas. Marquer deux fois le même ensemble produit l'état défini sans doublon d'événement et les champs réservés au mail restent absents du DTO.

Références de l'analyse

[monorepo](#) · [services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/notification/controller/NotificationController.kt:29](#) Routes personnelles, pagination et lecture.

[monorepo](#) · [services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/notification/controller/dto/MarkNotificationAsReadDTO.kt:7](#) Liste non vide et date de lecture.

[monorepo](#) · [services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/notification/repository/NotificationRepositoryJPAImpl.kt:35](#) Filtrage par abonné pour page et mutation.

monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/notification/controller/dto/PayloadDTO.kt:20
Projection et exclusions de payload.

NC-09 **Cible non qualifiée**

Rendre les trois emails métiers localisés et les remettre au SMTP

COMPORTEMENT OBSERVÉ

L'implémentation nommée EmailSendinBlueImpl utilise JavaMailSender et SMTP ; le code lu n'appelle pas une API Sendinblue. MessageBuilder construit un message HTML avec enveloppe générique et template FreeMarker, puis localise sujet et corps en FR, EN ou DE. Il ne résout que PROJECT_USER_MEMBERSHIP_ADDED, DATAFLOW_EXECUTION_FAILED et DATASET_QUALITY_FAILED. Pour toute autre action, il retourne null et l'émetteur saute l'envoi. DATASET_COLLECT_FAILED est donc souscriptible en EMAIL dans l'API mais n'a pas de template résolu ici. Le modèle est construit depuis les ActionObjects JSON, avec alias de noms et contexte de client ; le champ mailContent accumulé n'est pas utilisé comme corps par ce builder.

UTILISATEURS ET RÔLES

Utilisateurs destinataires d'invitations projet, échecs de flow et erreurs de qualité ; opérateurs mail.

INTERFACES DANS LE CODE

EmailSendinBlueImpl ; MessageBuilder.build/resolveTemplate ; EmailArgumentMapper ; templates/mail/*.ftlh.

DONNÉES ET ÉTATS

Locale du snapshot abonné ; destinataires EMAIL ; modèles et messages localisés en ressources applicatives ; configuration d'expéditeur.

DÉPENDANCES ET ÉCHANGES

Serveur SMTP, JavaMail, FreeMarker et bundle de traductions ; URLs fournies dans le payload métier.

COMPORTEMENT CIBLE PROPOSÉ

Notifications possède un renderer Rust à templates versionnés et des adaptateurs d'effet mail. Le catalogue affiche les canaux réellement disponibles. Les liens, champs obligatoires et échappements HTML sont validés ; Egress contrôle la destination SMTP et les références externes.

MIGRATION DE CETTE CAPACITÉ

Reprendre les trois modèles et leurs traductions sous tests de rendu. Décider explicitement de créer un email d'échec de collecte ou de retirer ce canal de cette action. Distinguer absence de template, canal désactivé et remise au fournisseur dans les reçus.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Chaque combinaison des trois actions et trois langues produit un sujet et un corps attendus sans champs manquants. Une action sans template donne un statut unsupported observable. Un nom contenant du HTML est échappé et un refus SMTP laisse une tentative rejouable.

Références de l'analyse

monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/notification/service/notification/email/MessageBuilder.kt:30 Absence de template retourne null.

monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/notification/service/notification/email/MessageBuilder.kt:78 Modèle depuis ActionObjects.

[monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/notification/service/notification/email/MessageBuilder.kt:115](#) Trois actions et langues résolues.

[monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/notification/service/notification/email/sendinblue/EmailSendinBlueImpl.kt:11](#) Adaptateur JavaMailSender et saut des messages sans template.

NC-10 **Cible non qualifiée**

Authentifier les connexions Socket.IO et borner leur durée

COMPORTEMENT OBSERVÉ

Le transport temps réel est Socket.IO/Engine.IO, pas un WebSocket JSON interchangeable. Le service authentifie lui-même ses connexions : payload auth, header Authorization Bearer, puis cookie legacy si l'option le permet. Les paramètres de query ne servent pas de jeton. Avec le fallback cookie désactivé, une connexion peut attendre le paquet CONNECT pour son payload auth, mais aucune action de room n'est admise avant authentification. Le token doit être actif, décodable et porteur d'un sub. Une gate de protocole refuse les CONNECT invalides avant leur acquittement ; un délai d'authentification ferme les connexions incomplètes et l'expiration du JWT provoque aussi la fermeture même au repos. Les frames mal formées sont fermées et les données client non authentifiées ne sont pas écrites dans les logs ordinaires.

UTILISATEURS ET RÔLES

Frontends et clients Socket.IO authentifiés.

INTERFACES DANS LE CODE

Handshake Engine.IO ; paquet Socket.IO CONNECT avec auth ; TokenAuthService ; SocketIoConnectGate ; événements join-room/leave-room.

DONNÉES ET ÉTATS

Sessions en mémoire, sujet et JWT validé, échéances d'authentification et d'expiration ; aucune session temps réel durable.

DÉPENDANCES ET ÉCHANGES

JWT decoder/IdP, configuration cookie fallback, APISIX sur une route sans authentification de gateway, netty-socketio.

COMPORTEMENT CIBLE PROPOSÉ

Un serveur Rust compatible avec le protocole réellement utilisé, ou une transition de clients vers un protocole explicite, reprend les mêmes refus et échéances. IAM émet les tokens sans Keycloak JVM ; la durée de socket ne dépasse pas celle du credential.

MIGRATION DE CETTE CAPACITÉ

Exécuter le corpus de tests wire contre le serveur candidat avec les versions de clients déployées. Retirer le cookie legacy après migration des clients et fournir un renouvellement/reconnect documenté. Préserver la séparation entre authentification du transport et autorisation des rooms.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un jeton forgé ou expiré n'obtient aucun CONNECT réussi ; un jeton en query est ignoré. Un client sans auth ferme à échéance. Un socket valide devient inutilisable à l'expiration même sans nouveau message, et le frontend peut se reconnecter avec un token renouvelé.

Références de l'analyse

[monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/synchronousCommunication/service/TokenAuthService.kt:28](#) Sources de credentials et ordre de résolution.

monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/synchronousCommunication/service/SocketIoService.kt:292 Délai d'authentification et échéance du JWT.

monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/synchronousCommunication/service/SocketIoService.kt:574 Refus du CONNECT portant un payload invalide.

monorepo · services/notification-center/AGENTS.md:1 Contrat de gateway et tests wire à préserver.

NC-11 Cible non qualifiée

Autoriser les namespaces et les rooms avant chaque diffusion

COMPORTEMENT OBSERVÉ

Le service prévoit OFF, SHADOW et ENFORCE pour l'autorisation Socket.IO, avec OFF comme valeur par défaut dans les propriétés. En ENFORCE, une réponse IAM manquante refuse l'action. Le namespace racine donne la room personnelle correspondant au sujet ; /admin exige le droit administrateur système. Un namespace projet doit contenir un UUID canonique et ses rooms dataflow-exec-* sont soumises au can_view du projet. /collect accepte des rooms collect-* contrôlées sur la datasource. Les familles inconnues sont refusées en mode imposé. Le serveur revalide le credential et l'autorisation pour chaque client avant diffusion et retire les clients refusés de la room. Les namespaces projets dynamiques ont un plafond et une éviction après inactivité.

UTILISATEURS ET RÔLES

Utilisateurs de projets, administrateurs de clusters, créateurs de sources et opérateurs IAM.

INTERFACES DANS LE CODE

WsAuthorizationService.authorizeConnect/authorizeJoin ; SocketIoService.onJoinRoom/publish ; namespaces /, /admin, /collect et /projects:{projectId}.

DONNÉES ET ÉTATS

Appartenance aux rooms en mémoire ; réservations de namespaces dynamiques, échéances d'inactivité et cache éventuel du client IAM.

DÉPENDANCES ET ÉCHANGES

IAM can_view pour projet/datasource et privilège système administrateur ; WsAuthzProperties.

COMPORTEMENT CIBLE PROPOSÉ

Notifications impose une ACL par famille de message et ressource canonique. Un événement de flow prouve son appartenance au projet demandé. Les décisions sont revérifiées à la livraison avec une politique de cache bornée ; Audit trace les refus sans exposer les payloads.

MIGRATION DE CETTE CAPACITÉ

Comparer les décisions shadow aux routes réellement utilisées, résoudre les UUID legacy, puis activer ENFORCE. Reprendre les limites de namespaces et définir la fréquence maximale de revalidation IAM pour la charge cible.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Après révocation, un client connecté ne reçoit plus les messages protégés. Une room d'exécution d'un autre projet est refusée même si l'utilisateur connaît son ID. Une rafale de namespaces invalides ne dépasse ni le plafond mémoire ni le délai de nettoyage.

Références de l'analyse

monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/synchronousCommunication/service/WsAuthorizationService.kt:101 Plans de droits par namespace et room.

`monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/synchronousCommunication/service/SocketIoService.kt:218` Contrôle de chaque destinataire avant émission.

`monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/synchronousCommunication/service/SocketIoService.kt:629` Création bornée des namespaces projets et éviction.

`monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/configuration/WsAuthzProperties.kt:16` Modes d'autorisation et valeurs par défaut.

NC-12 Cible non qualifiée

Diffuser les six familles de messages de supervision et d'interaction

COMPOTEMENT OBSERVÉ

Le consumer de communication synchrone convertit six familles Protobuf. NOTIFICATION vise la room utilisateur à la racine, sous le nom notification. CLUSTER_SESSION_EXEC, CLUSTERS_SESSION_EXEC_LIST et CLUSTER_MONITORING visent /admin avec respectivement cluster-{id}, cluster-all et cluster-monitoring. DATAFLOW_REFRESH vise le namespace /projects:{projectId} et dataflow-exec-{executionId}. CHECK_FILE vise /collect et collect-{datasourceId}, avec résultat d'analyse de schéma. La publication utilise les clients actuellement présents dans les namespaces locaux. Ce chemin ne montre ni curseur client, ni journal de rattrapage, ni acquittement métier. Le consumer utilise un groupe partagé ; la portée de diffusion entre pods doit être testée puisque les sockets sont maintenues en mémoire locale et le chart autorise le scale.

UTILISATEURS ET RÔLES

Centre de notifications frontend, pages d'exécution des flows, administration de clusters et assistants d'import de fichiers.

INTERFACES DANS LE CODE

SynchronousCommunicationConsumer ; PublishMessageCommandHandler ; EnumSynchronousCommunicationType ; noms de rooms et événements ci-dessus.

DONNÉES ET ÉTATS

Messages éphémères issus du bus ; sessions/rooms en mémoire ; l'historique de notifications INAPP existe séparément en PostgreSQL.

DÉPENDANCES ET ÉCHANGES

Événements produits par data-collect/data-dev et autres services de runtime ; contrats Protobuf ; clients Socket.IO.

COMPOTEMENT CIBLE PROPOSÉ

Notifications transporte des projections de runs, de contrôles de fichiers et de supervision. Un identifiant, un numéro de séquence et une URL de resynchronisation permettent au frontend de rattraper une interruption. Les messages liés aux données référencent l'édition approuvée ; ils ne font pas foi comme checkpoint publié.

MIGRATION DE CETTE CAPACITÉ

Préserver les six payloads pendant la migration des clients. Définir une diffusion entre instances ou un routage garantissant l'accès aux sockets du destinataire. Séparer les flux de supervision volumineux des notices durables et leur rétention.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Avec deux instances et des clients répartis, chaque destinataire autorisé reçoit le message attendu. Après déconnexion, un frontend restaure l'état courant et détecte les séquences manquantes. Un CHECK_FILE ne révèle pas le schéma à un utilisateur sans droit sur sa source.

Références de l'analyse

`monorepo · services/notification-center/app/notification-center-domain/src/main/kotlin/com/cleyrop/domain/synchronousCommunication/command/PublishMessageCommandHandler.kt:29` Routage des six familles vers namespaces/rooms/events.

`monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/common/consumer/SynchronousCommunicationConsumer.kt:15` Consumer et groupe Kafka.

`monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/common/consumer/convert/SynchronousCommunicationEventProtoConverter.kt:16` Conversion des payloads.

`monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/synchronousCommunication/service/SocketIoService.kt:192` Diffusion sur les clients locaux présents.

NC-13 **Cible non qualifiée**

Exploiter la messagerie avec outbox, quarantaine et migrations

COMPOTEMENT OBSERVÉ

Le relais d'outbox sélectionne les messages non envoyés avec FOR UPDATE SKIP LOCKED, attend la confirmation Kafka puis renseigne sentAt ; erreurs et tentatives restent persistées. Le nettoyage ne retire que les messages envoyés au-delà de la rétention. Les consommateurs utilisent un gestionnaire d'erreur avec reprises puis DLT ; une DLT n'est jamais republiée récursivement en DLT. Le consumer terminal persiste les coordonnées et l'erreur, avec un UUID stable dérivé du topic/partition/offset de la DLT, et réémet l'erreur si PostgreSQL échoue. Un indicateur de santé expose les listeners arrêtés de manière anormale. Les migrations Flyway coexistent avec des migrations de code jouant des abonnements ; les instructions du composant signalent l'absence de verrou global des migrations de code et leur sortie de processus en cas d'échec.

UTILISATEURS ET RÔLES

SRE, opérateurs d'upgrade, support des événements rejetés et responsables de continuité de service.

INTERFACES DANS LE CODE

OutboxRelay ; OutboxEventJPA.claimUnsentBatch ; DeadLetterEventConsumer ; KafkaListenersHealthIndicator ; CleyropMigrationRunner.

DONNÉES ET ÉTATS

outbox_event, dead_letter_event, processed_message, historique Flyway et code_migration ; marqueurs de livraison et tentatives.

DÉPENDANCES ET ÉCHANGES

PostgreSQL, Kafka, chart Kubernetes, probes et metrics ; migrations de contrat de souscription.

COMPOTEMENT CIBLE PROPOSÉ

Un socle Rust commun fournit outbox, replay, dead-letter, health et métriques par backlog/âge. Le bus final est sans JVM et conserve les garanties nécessaires de rétention et d'ordre. Les migrations ont un verrou explicite, un statut durable et une procédure de rollback Expand/Migrate/Contract.

MIGRATION DE CETTE CAPACITÉ

Reprendre les messages non envoyés, les marqueurs et la quarantaine avant bascule de consumer. Établir le plan de replay et les clés d'idempotence. Transférer les migrations d'abonnements dans des jobs bornés reprenables, puis retirer Spring, Kafka JVM et leurs images.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une interruption entre acquittement du bus et commit provoque au plus un replay absorbé. Une DLT illisible ou une panne DB n'avance pas l'offset terminal. Un upgrade concurrent n'exécute pas deux fois une migration et une restauration remet en cohérence inbox/outbox/reçus.

Références de l'analyse

`monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/common/outbox/OutboxEventJPA.kt:14` Claim verrouillé de l'outbox.

`monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/common/outbox/OutboxRelay.kt:23` Acquittement puis état envoyé.

`monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/common/deadletter/DeadLetterEventConsumer.kt:26` Persistance terminale et comportement en erreur.

`monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/codemigration/migrations/SubscriptionMigration.kt:20` Replay des souscriptions par migration de code.

Catalogue des actions de notification

Les actions réservées APPLICATION_USER_MEMBERSHIP_ADDED et APPLICATION_DATASET_LINK_FAILED ne sont pas des valeurs actives. INAPP dépend de la présence d'un événement, d'un abonné, d'un canal sélectionné et des décisions d'autorisation ; le tableau ne promet pas une émission pour chaque nom.

Action	Déclarée en protobuf	Création via DTO abonnement	Template email résolu
DATASET_COLLECT_FAILED	Oui	Oui	Non
DATASET_DATAFLOW_FAILED	Oui	Non	Non
DATASET_DELETED	Oui	Non	Non
DATASET_CATALOG_PUBLISHED	Oui	Non	Non
DATASET_MANUALLY_REFRESHED	Oui	Non	Non
DATASET_QUALITY_FAILED	Oui	Oui	Oui
DATASET_OWNER_PROMOTED	Oui	Non	Non
PROJECT_USER_MEMBERSHIP_ADDED	Oui	Non	Oui
PROJECT_DATASET_REQUESTED	Oui	Non	Non
PROJECT_DATASET_ACCEPTED	Oui	Non	Non
PROJECT_DATASET_REFUSED	Oui	Non	Non

Action	Déclarée en protobuf	Création via DTO abonnement	Template email résolu
PROJECT_DATASET_DELETE D	Oui	Non	Non
PROJECT_DATASOURCE_REQUESTED	Oui	Non	Non
PROJECT_DATASOURCE_APPROVED	Oui	Non	Non
PROJECT_DATASOURCE_REFUSED	Oui	Non	Non
PROJECT_OWNER_PROMOTED	Oui	Non	Non
DATAFLOW_MR_REQUESTED	Oui	Non	Non
DATAFLOW_MR_ACCEPTED	Oui	Non	Non
DATAFLOW_MR_REFUSED	Oui	Non	Non
DATAFLOW_EXECUTION_FAILED	Oui	Oui	Oui
DATAFLOW_OWNER_PROMOTED	Oui	Non	Non
APPLICATION_GROUP_MEMBERSHIP_ADDED	Oui	Non	Non
GIT_MR_REQUESTED	Oui	Non	Non
GIT_MR_ACCEPTED	Oui	Non	Non
GIT_MR_REFUSED	Oui	Non	Non
DATASOURCE_DELETED	Oui	Non	Non
DATASOURCE_OWNER_PROMOTED	Oui	Non	Non

Architecture et exploitation du service cible

Plan de contrôle Rust proposé

- Notifications Rust possède le registre des actions, les schémas de payload, les canaux réellement supportés, les préférences par sujet et la boîte personnelle. PostgreSQL reste autoritaire pour ces commandes et états d'effets.
- IAM décide l'accès à la ressource lors de la souscription et avant effet. Les notices de suppression et de demandes d'accès ont une politique dédiée, sans supposer que la ressource reste consultable.
- Les événements annonçant une donnée publiée référencent l'édition Iceberg approuvée, le run et la publication. Une réussite de transport, de collecte ou de notification n'est pas un acquittement de publication.
- Audit corréle événement source, regroupement, décision IAM, rendu versionné, canal, tentatives et reçu. Une livraison incertaine reste représentée comme telle.

- Les templates, canaux et locales sont versionnés ; le frontend n'offre que les combinaisons supportées. Les actions automatiques mentionnent leur producteur et leur mandat de destinataire.

Plan de données proposé

- Workers Rust bornés pour fanout, regroupement, rendu et effets ; taille des payloads, nombre de destinataires, débit et délai bornés par politique tenant.
- Adaptateur SMTP ou fournisseur encapsulé avec identifiant d'effet stable et traitement de l'incertitude ; aucun engagement implicite d'exactly-once SMTP.
- Service temps réel sans JVM avec compatibilité Socket.IO validée ou migration coordonnée des clients ; authentification à échéance, droits de room et diffusion entre instances.
- Bus durable sans JVM, outbox et consommateurs idempotents. Les messages de supervision peuvent être compacts, mais un état durable consultable permet la resynchronisation après une coupure.
- Le transport des notifications ne publie aucune table Iceberg. Les références de données pointent sur les publications approuvées et les lecteurs restent épinglés à leur édition.

Découpage du core Rust

- notifications-contracts : registre action/version, schémas, ressources canoniques, canaux, templates et validation des payloads.
- notifications-preferences : miroir IAM, abonnements personnels/automatiques, préférences par canal et commandes idempotentes.
- notifications-aggregation : groupes déterministes, fenêtres bornées, claim concurrent et publication atomique du résultat vers l'outbox.
- notifications-delivery : vérification IAM avant effet, rendu HTML localisé, adaptateurs d'envoi et machine d'état des reçus.
- notifications-inbox : boîte personnelle, pagination, lecture et rétention ; effets séparés de la projection temps réel.
- notifications-realtime : protocole client, authentification, ACL de rooms, gestion de l'expiration, limites de namespaces, diffusion entre instances et resynchronisation.
- Socle partagé d'outbox, processed-message durable, dead-letter, métriques, replay et migrations ; remplacement du runtime JVM et du broker JVM coordonné avec les producteurs.

Identité, autorisation et isolation

- HTTP : principal courant pour boîte et souscriptions ; confiance de X-Userinfo conditionnée au mode gateway et au proxy déclaré. Les routes nouvelles doivent passer le garde de couverture du composant.
- Socket : auth payload/header Bearer, cookie legacy sous option, jamais de token en query. La gate CONNECT, le timeout d'auth et l'échéance du token sont des garanties distinctes à conserver.
- Autorisation de livraison et de rooms sous OFF/SHADOW/ENFORCE ; défaut OFF dans les propriétés lues. Ne pas déduire le mode réellement déployé du seul chart ou du code.
- Le contrôle de livraison vise actuellement les trois actions souscriptibles par REST et s'effectue avant l'accumulation. La cible doit trancher la révocation entre cette étape et l'effet SMTP ou INAPP.

- Le contrôle projet des rooms d'exécution ne constitue pas à lui seul une preuve que l'exécution appartient au projet. Un scénario de croisement des identifiants doit caractériser la protection de bout en bout.
- Les emails et payloads de notification peuvent porter des noms et liens métier ; minimiser les snapshots, borner la rétention des dead-letter et restreindre les accès aux journaux.
- Les namespaces dynamiques sont validés, plafonnés et évincés ; les journaux ordinaires excluent les valeurs client non authentifiées. Le rendu HTML doit conserver l'échappement et les contrôles de liens.

Exploitation et reprise

- Deux modules Gradle, démarrage Spring, Flyway et migrations de code ; le runner de migrations de code ne prend pas de verrou global et les instructions signalent un arrêt avec code 0 en cas d'échec.
- Outbox réessayée et purgée seulement après envoi ; un crash après confirmation Kafka peut entraîner une rediffusion. Les consommateurs doivent conserver une idempotence compatible avec la rétention de replay.
- DLT terminale persistée avant l'acquittement ; déduplication stable par coordonnées de DLT dans notification-center. Rejouer une DLT doit garder l'identité source et enregistrer l'opérateur.
- Indicateur de santé des listeners Kafka ; compléter par âge du plus ancien travail, taille de backlog, échecs IAM, rendus sans template, remises SMTP incertaines et clients temps réel rejetés.
- Le chart applique utilisateur non root, probes, références de secrets et options de volume. Le NetworkPolicy lu restreint l'entrée mais ne définit pas une restriction d'egress pour SMTP/IAM/bus.
- L'autoscaling est conditionné par la configuration de persistance ; la diffusion aux sockets et la réduction des accumulations doivent être qualifiées à plusieurs instances avant d'en déduire une scalabilité fonctionnelle.
- Les tests d'infrastructure demandent Docker et des conteneurs PostgreSQL/Kafka. Les tests existants couvrent notamment les canaux, duplications, templates, contrôle IAM et auth wire ; ils n'ont pas été exécutés dans cet audit.
- Detekt est configuré pour réécrire les fichiers : il n'a pas été lancé lors de cette lecture seule. Le garde des routes est présent dans la CI locale du service mais les instructions indiquent qu'il n'est pas inclus par la pipeline racine.

Séquence de migration

1. Exporter un inventaire anonymisé des abonnés, abonnements, médias, langues, actions utilisées et canaux effectivement livrés ; comparer aux 27 actions déclarées et aux trois templates.
2. Figurer les contrats HTTP, Protobuf, JSON interne et Socket.IO, avec fixtures et noms d'événements. Caractériser les lots bulk, suppression croisée et préférences médias sans supposer une correction compatible.
3. Rejouer le miroir IAM en parallèle sous indicateurs, réconcilier les identifiants, puis choisir une seule autorité de mise à jour et préparer la sortie de Keycloak.
4. Déployer le noyau Rust des préférences et de la boîte sur schéma étendu, migrer les UUID et dates, comparer les lectures puis basculer les commandes.
5. Migrer accumulations, outbox, marqueurs et quarantaines avec clés stables. Valider concurrence, crashes et replays avant bascule du regroupement et des effets.
6. Reprendre les rendus FR/EN/DE et qualifier les trois actions SMTP. Décider explicitement du cas EMAIL pour DATASET_COLLECT_FAILED et des autres actions sans template.

- 7. Qualifier le serveur temps réel et la diffusion entre instances avec les clients réels ; migrer les clients si le protocole change et imposer les ACL après observation shadow.
- 8. Retirer le bus et les composants JVM quand tous les producteurs/consommateurs sont basculés. Appliquer Expand/Migrate/Contract séparément, restaurer un backup complet en exercice et retirer les anciens secrets seulement après vérification.

Écarts et décisions encore ouverts

- Le mapping EntityActionTypeDAO partage mediaTypes par nom d'action. Reproduire deux abonnés avec canaux différents avant de conclure au comportement persistant et de choisir la conversion cible.
- La suppression par listes séparées et la déduplication du bulk changent la correspondance des couples fournis. Aucun incident runtime n'est établi ici ; caractériser les attentes des clients et les lots croisés.
- La déduplication d>ActionResource s'appuie sur les accumulations encore présentes. Quel horizon de replay est garanti après leur suppression ? Prévoir un registre durable dans la cible.
- Le réducteur ne montre pas un claim ou une transaction englobant sélection/publication/suppression. Confirmer par un scénario à deux instances et crash après outbox avant d'affirmer une duplication réelle.
- Le consumer de communication synchrone partage un groupe alors que les sockets sont locales. Vérifier les mécanismes de routage/broadcast effectivement déployés et la livraison sur deux pods.
- MessageBuilder ne rend pas DATASET_COLLECT_FAILED malgré la sélection EMAIL possible. Faut-il créer un template, corriger l'offre de canal ou conserver temporairement ce comportement ?
- NotificationDAO déclare une suppression SQL visant subscriber alors que son entité correspond à notification. Aucune route de suppression n'est exposée dans les controllers lus ; vérifier les chemins réellement appelés avant de traiter cette observation comme un défaut runtime.
- Quels producteurs émettent encore les actions Git et les actions de merge request de flow ? Le Protobuf les conserve mais une migration retire GIT_MR_REQUESTED des abonnements.
- Le consumer historique Subscriber et le nouveau consumer IAM peuvent coexister. Définir l'ordre, la source autoritaire et la résolution d'une adresse ou activation contradictoire.
- Quel délai maximal est accepté entre retrait d'un droit et un email déjà préparé ? Le contrôle actuel est placé avant accumulation et non juste avant l'appel SMTP.
- La sortie finale de JVM inclut le serveur Socket.IO, le bus Kafka, FreeMarker, JavaMail et Keycloak, pas uniquement les handlers Kotlin. Aucun substitut n'est déclaré qualifié par cet audit.

Références de l'analyse

[monorepo · services/notification-center/AGENTS.md:28](#) Contrats d'auth, indicateurs, outbox et contraintes d'exploitation.

[monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/notification/controller/NotificationController.kt:29](#) Surface de boîte personnelle.

[monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/subscription/controller/SubscriptionController.kt:49](#) Surface de souscription.

[monorepo · services/notification-center/chart/templates/deployment.yaml:50](#) Conteneur, probes, secrets et volumes.

[monorepo · services/notification-center/chart/templates/networkpolicy.yaml:23](#) Contrôle réseau entrant.

[monorepo · services/notification-center/app/notification-center-infrastructure/src/main/kotlin/com/cleyrop/domain/notificationcenter/notification/repository/dao/NotificationDAO.kt:35](#) Mapping et annotation de suppression à caractériser.